

AfriShield ML Evolution

A beginner's guide — from a hand-written formula to a real, tested improvement

This is the story of how AfriShield's flood-risk prediction went from a simple formula, to a machine-learning model trained on made-up data, to one trained on real historical floods and genuinely improved. No prior ML knowledge needed — every technical term is explained as it comes up, and defined again in the glossary at the end.

CHAPTER 1

The simple version (no ML at all)

Before any machine learning, there was just a formula a person wrote by hand: take the rainfall amount and the river level, average them together, and sort the result into "low," "medium," or "high" risk. Simple, and easy to explain to anyone — but it's just a fixed recipe. It doesn't "learn" anything; a human decided the rule in advance.

CHAPTER 2

Teaching a computer to guess (the first ML attempt)

We wanted a second opinion — something that learned a pattern by example instead of following a hand-written rule. But there was a problem: we didn't have real historical records of "it rained this much, the river was this high, and yes/no a flood happened." So we made up 2,000 fake example situations using code, decided what the "correct answer" should be for each one, and taught a simple model (**Logistic Regression** — see glossary) to recognize the pattern.

This worked, technically — the model got good at predicting the fake examples. But it never proved anything about the real world, because it never saw the real world.

CHAPTER 3

The moment of truth — checking against real floods

Later, we found a real historical record: the Dartmouth Flood Observatory, a genuine database of confirmed real floods going back decades. We used it to put both models to a real test — not "did they get the fake quiz right," but "would they have actually caught real floods that really happened?"

The honest result was humbling: on 239 real, confirmed flood-days, the simple hand-written formula caught **41%** of them (this is called **recall** — see glossary). The fancier trained ML model, trained on fake data, only caught **28%**. The "smarter-looking" approach was actually worse in the real world, because it had only ever practiced on made-up questions. A model is only as good as what it learned from, no matter how sophisticated the technique.

The real database gave us something new: **40,904 real days** of actual rainfall and river data, with **239 of those days** being real confirmed flood events. This time, instead of just using the real data to check the model afterward, we used it to actually teach the model from scratch.

One technical wrinkle: the real river data was measured in "discharge" (cubic meters of water flowing per second) — a different unit than the "river level in meters" our system actually uses. We couldn't directly convert one to the other, so we ranked each day's discharge against that same river's own history (e.g., "higher than 80% of all days ever seen on this river") and used that ranking as a stand-in for river level. It's an approximation, not a perfect conversion — and we say so honestly.

Another real wrinkle: floods are rare — only 239 of 40,904 days (about half a percent) were actual flood days. Left alone, a model could just learn to always guess "no flood" and still be right 99%+ of the time — technically accurate, but useless, since it would never warn anyone. The fix is a setting called **class weight balancing** (see glossary): it tells the model to pay extra attention to the rare flood examples instead of ignoring them.

The result, tested fairly on a chunk of real data the model never trained on (called **held-out test data** — see glossary): recall jumped to **81%** — three times better than the old fake-data model's 27%, and well ahead of the hand-written formula's 48% on that same test slice.

Catching more real floods came with a cost: the model also cried "flood!" on more days that were actually fine (a **false positive** — see glossary), raising the false-alarm rate from about 14% to about 34%. This is a completely normal, expected trade-off in machine learning — a model this alert can't also be perfectly picky.

The fix is called **threshold tuning** (see glossary). The model doesn't output a flat yes/no — it outputs a confidence number, like "I'm 73% sure this is a flood risk." Normally we just say "if it's over 50% confident, call it a risk." But that 50% cutoff is just a default, not a law of nature — we can slide that dial up or down and measure what happens at each setting, like a report card across the whole range.

The first pass at this looked excellent: at a confidence cutoff of 80% instead of 50%, we measured 62.5% recall and only a 17.7% false-alarm rate. But before trusting that number, we double-checked how it was measured — and found a real bug (next chapter).

Real floods usually last several days in a row, and our real dataset labels every one of those days. The first "81% recall" measurement split the data into a training pile and a testing pile *by individual day*, at random. That sounds fair, but it quietly let some days of the SAME flood event end up in training while other days of that exact same event ended up in testing — like letting a student see three questions from a five-question quiz beforehand. This is called **data leakage** (see glossary), and checking for it turned up a real problem: **19 of our 29 real flood events (65%) had days on both sides of the split.**

The honest fix is called **leave-one-event-out cross-validation** (see glossary): instead of one split, we ran the training process 29 separate times, each time hiding one whole flood event the model had never seen any part of, and pooling all 29 rounds of results together. That way, every single real event gets to be "the surprise test" exactly once, with zero chance of leakage.

The corrected, trustworthy number: at threshold 0.80, the model catches **52.7%** of real floods with a **17.95%** false-alarm rate — still genuinely better than the hand-written formula's 41.0% recall / 22.05% false-alarm rate on the same events, just a smaller win than the leaked measurement first suggested. Catching a bug like this before shipping it is exactly what "held-out test data" and careful measurement are for.

With the leakage bug fixed, the next step was to actually put the improved model into the live system and retrain it one final time on all the real data available. Before calling that done, we ran one more sanity check: feed it the ten sample cities our own demo uses (Lagos, Kampala, Cairo, and so on) and see what it says.

It said **"low risk" for all ten** — including Lagos and Kampala, which the hand-written formula correctly flags as "high risk," and which the whole demo is built around showing as dangerous. Something was wrong, even though every step up to this point had been measured correctly.

The cause: real confirmed floods are extremely rare in 26 years of history (about 1 in every 170 days), so the model's real, learned sense of "elevated" river conditions is far more extreme than the sample numbers our team picked by hand for the demo's ten cities — those were chosen to look sensible next to the simple 50/50 formula, not against this model's real-world scale. Two numbers that share the same 0-4 range on paper turned out to mean very different things. This is called a **calibration mismatch** (see glossary), and it's a completely different, more fundamental problem than a coding bug — the model itself is honestly measured as better on real history, but it can't yet be safely pointed at this project's own hand-picked demo inputs.

So we reverted: the live system still runs the original fake-data-trained model today, on purpose, rather than shipping something that would make the demo look broken. The real-data model and its genuine 52.7%-recall improvement are proven and kept for the record — they're just not turned on yet, until someone reconciles the demo's sample numbers with the real scale the model actually learned. Catching this before the actual presentation, instead of during it, is the whole point of testing honestly at every step.

The full journey, in one table

Version	Trained on	Recall (catches real floods)	False alarms
Hand-written formula	(no training, just math)	41–48%	22%
First ML model	fake, made-up data	27–28%	14%

New ML model (first measurement)	real historical floods	81–87% (<i>leaked, see Ch. 6</i>)	34–42%
New ML model (leak-free, tuned)	real historical floods	52.7%	17.95%

The leak-free, tuned version is a genuine, honestly-measured improvement over everything that came before it. It is **not yet switched on in the live system**, though — Chapter 7 explains the separate calibration problem that's holding that back, independent of how well the model itself measures.

Glossary

Training data	The example situations used to teach the model the pattern — like practice questions a student studies from.
Test data (a.k.a. "held-out" data)	Examples deliberately hidden from training, saved to check afterward whether the model actually learned something useful — or just memorized the practice questions. This is the only fair way to know if a model really works.
Recall	Out of all the real flood days that actually happened, what percentage did the model correctly flag? High recall means the model rarely misses a real flood.
False positive	A day the model flagged as risky that turned out to be perfectly safe — a false alarm.
False-positive rate (FPR)	Out of all the days that were actually safe, what percentage did the model incorrectly flag as risky? Lower is better — it means fewer unnecessary alarms.
Precision	Out of all the days the model flagged as risky, what percentage were real floods? Recall and precision are often in tension — being very cautious (high recall) usually means more false alarms (lower precision).
Threshold	The confidence cutoff used to turn a model's "I'm 73% sure" into a decision ("yes, warn" or "no, don't"). Sliding this cutoff up or down trades recall against false alarms.
Class imbalance	When one outcome (like "no flood") is far more common in the data than another (like "flood"), making it easy for a lazy model to just always guess the common one and still look accurate.
Class weight balancing	A training setting that tells the model to pay extra attention to the rare, important category (floods) instead of letting it get ignored because there's fewer examples of it.
Logistic Regression	A simple, well-understood method for sorting examples into categories by finding the best dividing line between them. Chosen here because it's easy to explain, not because it's the most powerful option available.
Overfitting	When a model gets very good at its exact practice examples but fails on new, real situations — closely related to Chapter 2's lesson: performing well on fake data proves nothing about the real world.
Youden's J statistic	A simple way to combine "how many did we catch" (recall) and "how many false alarms" (false-positive rate) into one number, used to help compare different threshold settings at a glance.
Data leakage	When information that should only be visible during testing accidentally sneaks into training too — like a student seeing part of the answer key beforehand. It makes a model look better than it really is, until someone checks for it.

**Leave-one-event-out
cross-validation**

A leak-proof way to test a model when you don't have much real data: instead of one train/test split, retrain many times, each time hiding one whole real event completely, then combine every round's results. Slower, but trustworthy.

Calibration (mismatch)

Whether a model's numbers mean the same real-world thing as the numbers being fed into it. Two values can share the same numeric range (like 0-4) and still mean very different things if one was picked by hand to "look sane" and the other was learned from a completely different real-world distribution.